

SOS-BESOIN

Plateforme web transactionnelle de demandes urgentes

Cours	420-951-MA - Developpement Web 4 (Hiver 2026)
Laboratoire	Lab 2 - Server Actions, API Routes & Panier
Livrable	Livrable 1 - Document PDF
Equipe	SuperMoms
Sujet du projet	SOS-BESOIN - marketplace de demandes urgentes
Date de remise	27 avril 2026

Membres de l'equipe

Marina Kamel	Server Actions + Zod + Formulaires (Epic A)
Asma Ajroudi	API Routes + Postman (Epic B.1)
Sonia Corbin	Requetes Prisma avancees + Pagination + Agregation (Epic B.2/B.3)
Sonia Mhimdi	Panier client persistant + UX (Epic C)

Lien vers le depot GitHub (public, cliquable) :

<https://github.com/MarinaKamel-coder/sos-besoin>

Table des matieres

1. Presentation du projet
2. Lien vers le depot GitHub
3. Server Actions implementees
4. API Routes RESTful
5. Captures d'ecran - Tests Talend API Tester
6. Fonctionnement du panier
7. Captures d'ecran - Interface utilisateur
8. Bonus - Pagination par curseur

1. Presentation du projet

SOS-BESOIN est une plateforme web transactionnelle qui permet a des **clients** de publier des demandes urgentes et a des **prestataires** de soumettre des offres. Le client peut accepter une offre, l'ajouter a son **panier** et confirmer la reservation via un flux de paiement (Stripe en mode test) avec integrite transactionnelle (ACID) gere par Prisma.

Stack technique

Framework	Next.js 16.2 (App Router)
Langage	TypeScript
Base de donnees	PostgreSQL (Neon)
ORM	Prisma 7.8
Authentification	Clerk (@clerk/nextjs ^7.0)
Validation	Zod
Style	Tailwind CSS v4
Paieement	Stripe (test mode) - prevu

Modele de donnees principal

Le schema Prisma definit les entites suivantes : User, Profile, Category, ServiceRequest, RequestCategory (table de jointure N-N), Offer, **Cart**, **CartItem**, Booking, Payment et AdminAction. Les modeles ServiceRequest et Offer integrent un champ version (Int) qui implemente un verrouillage optimiste.

2. Lien vers le depot GitHub

Le depot est **public** et accessible a l'adresse suivante (URL cliquable) :

<https://github.com/MarinaKamel-coder/sos-besoin>

Contributions individuelles (git log). Chaque membre de l'equipe a des commits visibles dans l'historique du projet.

3. Server Actions implementees

Les Server Actions (Next.js App Router) sont declarees avec "use server" et appelees directement depuis les composants client via useActionState. Toutes les entrees sont validees par un schema Zod avant ecriture en base.

3.1 Profil utilisateur (src/action/userActions.ts)

Action	Role	Schema Zod
getMyProfileAction()	Recupere le profil de l'utilisateur Clerk connecte.	-
updateProfileAction(state, formData)	Upsert du profil utilisateur connecte.	profileUpdateSchema
deleteAccountAction()	Supprime le compte utilisateur (cascade Prisma).	-

3.2 Demandes (src/action/requestActions.ts)

Action	Role	Schema Zod
getRequestsAction(filters?)	Liste les demandes (avec categories et nombre d'offres).	
createRequestAction(state, formData)	Cree une demande pour le client connecte.	requestCreateSchema
updateRequestAction(state, formData)	Met a jour avec verrou optimiste (version).	requestUpdateSchema
deleteRequestAction(id)	Supprime une demande dont l'utilisateur est proprietaire.	

Verrouillage optimiste. updateRequestAction utilise prisma.serviceRequest.updateMany avec un filtre where: id, version et un increment de version. Si result.count est 0, on retourne un message de conflit.

3.3 Offres (src/action/offerActions.ts)

Action	Role	Schema Zod
getOffersByRequestAction(requestId)	Liste les offres d'une demande, tri par prix.	-
createOfferAction(state, formData)	Cree une offre. Unicite (requestId, providerId).	offerCreateSchema
updateOfferAction(state, formData)	Met a jour prix/message avec verrou optimiste.	offerUpdateSchema
deleteOfferAction(id, version)	Retire une offre du prestataire (verrou optimiste).	(version)

3.4 Panier (src/action/cart.ts)

Action	Role	Schema Zod
getCart()	Recupere le panier de l'utilisateur connecte (avec items).	
addToCart(offerId)	Ajoute une offre au panier (unicite cartId+offerId).	(verifications)

updateCartItemQuantity(itemId, qty)	Met a jour la quantite d'un item.	(qty >= 1)
removeFromCart(itemId)	Retire un item du panier.	-
clearCart()	Vide entierement le panier (deleteMany).	-
confirmCart()	Transforme les items en bookings (transaction Prisma).	

4. API Routes RESTful

Les API Routes (Next.js App Router) exposent les ressources principales en REST. Chaque endpoint valide les payloads avec Zod et renvoie des codes HTTP standards.

Endpoint	Methodes	Description
<code>/api/service-requests</code>	GET, POST	GET liste les demandes (avec client, categories, _count). POST cree une demande validee par Zod.
<code>/api/service-requests/[id]</code>	GET, PUT, PATCH, DELETE	GET detail (offers + booking + payment). PUT remplacement complet. PATCH mise a jour partielle. DELETE refusee si une reservation est liee.
<code>/api/service-requests/[id]/offers</code>	GET, POST	GET liste les offres d'une demande. POST cree une offre pour cette demande.
<code>/api/offers/[id]</code>	GET, PATCH, PUT, DELETE	GET detail offre. PATCH change le statut (ACCEPTED / REJECTED / WITHDRAWN). PUT remplacement complet. DELETE refusee si l'offre est liee a un booking.

4.1 Exemple : POST /api/service-requests

Requete (body JSON)

```
{
  "title": "Reparation climatiseur urgent",
  "description": "Le climatiseur ne refroidit plus depuis hier soir.",
  "neededAt": "2026-12-31T12:00:00.000Z",
  "location": "Casablanca",
  "clientId": "cmolg8twg00028grsoc7gp9b5",
  "categoryId": "cmolg8u83000c8grsf24cqw2v"
}
```

Reponse 201 Created

```
{
  "id": "cmolh7j230000jwrs6yh272tu",
  "title": "Reparation climatiseur urgent",
  "status": "OPEN",
  "version": 1
}
```

5. Captures d'ecran - Tests Talend API Tester

Les API Routes du projet ont été validées à l'aide de **Talend API Tester** (extension Chrome). Les captures suivantes illustrent les principaux scénarios de test, incluant les cas **nominaux (200/201)** et les cas d'erreurs métier.

5.1 GET /api/service-requests

200 · Liste des demandes

Récupérer la liste complète des demandes avec leur client, leurs catégories et le nombre d'offres.

The screenshot displays the Talend API Tester interface. The top section shows the request configuration: METHOD is GET, URL is http://localhost:3000/api/service-requests, and the response length is 42 bytes. The bottom section shows the response details, including headers and the JSON body.

Request Configuration:

- METHOD: GET
- URL: http://localhost:3000/api/service-requests
- QUERY PARAMETERS: (None)
- HEADERS: (None)
- BODY: (None)

Response Details:

Status: 200 OK

Headers:

- vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
- content-type: application/json
- Date: Fri, 01 May 2026 15:17:13 GMT
- Connection: keep-alive
- Keep-Alive: timeout=5
- Transfer-Encoding: chunked

COMPLETE REQUEST HEADERS:

- sec-ch-ua-platform: "Windows"
- __internal-request-id: 25766609-30b0-4a60-8380-26fd1d0b6f0
- sec-ch-ua: "Google Chrome";v="147", "Not.A/Brand";v="8", "Chromium";v="147"
- sec-ch-ua-mobile: ?0
- Accept: */*
- Sec-Fetch-Site: none
- Sec-Fetch-Mode: cors
- Sec-Fetch-Dest: empty
- Sec-Fetch-Storage-Access: active
- Accept-Encoding: gzip, deflate, br, zstd
- Accept-Language: en-US,en;q=0.9,fr;q=0.8
- Cookie: __clerk_db_jwt_pvoN3Mew=dvb_3A21QK13M0axN1vH2fUgb7odA4a; __client_uat_ovoN3Mew=0 clerk_db_jwt_6e1dFHK=dvb_3B12wFNF5

Response Body (JSON):

```
[
  {
    id: "cmon222rx000fmcrrsyr1mowr6",
    title: "SOS: dépannage Wi-Fi (routeur + mesh)",
    description: "Instabilité Wi-Fi depuis une mise à jour. Besoin diagnostic et configuration stable.",
    neededAt: "2026-05-04T15:16:04.035Z",
    location: "Montréal (Rosemont)",
    status: "OPEN",
    version: 1,
    createdAt: "2026-05-01T15:16:04.365Z",
    updatedAt: "2026-05-01T15:16:04.365Z",
    clientId: "cmon2224s0002mcrrs5aabvqj8",
    client: {
      id: "cmon2224s0002mcrrs5aabvqj8",
      clerkId: "clerk_client_001",
      email: "client@sos-besoin.te
    },
    categories: [
      {
        requestId: "cmon222rx000fmcrrsyr1mowr6",
        categoryId: "cmon222fv000bmcrrs0fm3nycd",
        category: {
          id: "cmon222fv000bmcrrs0fm3nycd",
          name: "Tech",
          slug: "tech",
          createdAt: "2026-0
        }
      },
      {
        requestId: "cmon222rx000fmcrrsyr1mowr6",
        categoryId: "cmon222fv000bmcrrs0fm3nycd",
        category: {
          id: "cmon222fv000bmcrrs0fm3nycd",
          name: "Tech",
          slug: "tech",
          createdAt: "2026-0
        }
      }
    ]
  }
]
```

Capture : api-1-get-list.png

5.2 GET /api/service-requests/[id]

200 · Detail d'une demande

Detail complet d'une demande : client, categories, offres et booking lies.

The screenshot displays a REST client interface. The top section shows a GET request to the URL `http://localhost:3000/api/service-requests/cm0lg8ui2000e8grsafq6nybk`. The response status is **200 OK**. The response body is a JSON object containing details of a service request and its associated client.

Response Headers:

- vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
- content-type: application/json
- Date: Thu, 30 Apr 2026 12:48:23 GMT
- Connection: keep-alive
- Keep-Alive: timeout=5
- Transfer-Encoding: chunked

Response Body:

```
{
  "id": "cm0lg8ui2000e8grsafq6nybk",
  "title": "SOS: correction urgente (10 pages)",
  "description": "Besoin d'une correction orthographe/grammaire et mise en forme légère. Livraison demain soir.",
  "neededAt": "2026-05-01T12:17:42.297Z",
  "location": "En ligne",
  "status": "OPEN",
  "version": 1,
  "createdAt": "2026-04-30T12:17:42.586Z",
  "updatedAt": "2026-04-30T12:17:42.586Z",
  "clientId": "cm0lg8twg00028grsoc7gp9b5",
  "client": {
    "id": "cm0lg8twg00028grsoc7gp9b5",
    "clerkId": "clerk_client_001",
    "email": "client@sos-besoin.test",
    "name": "Camille Client",
    "role": "CLIENT",
    "createdAt": "2026-04-30T12:17:41.728Z",
    "updatedAt": "2026-04-30T12:17:41.728Z"
  },
  "categories": [

```

Capture : detail d'une demande.png

5.3 POST /api/service-requests

201 · Creation d'une demande

Creation d'une nouvelle demande validee par Zod ; statut initial OPEN, version 1.

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/api/service-requests
- Headers:** Content-Type: application/json
- Body:** A JSON object representing a service request:

```
{  "title": "Réparation climatiseur urgent",  "description": "Le climatiseur ne refroidit plus depuis hier soir.",  "neededAt": "2026-12-31T12:00:00.000Z",  "location": "Casablanca",  "clientId": "cmolgtwg00028grsoc7gp9b5",  "categoryId": "cmolgtwg00028grsf24cq2v"}
```
- Response:** 201 Created. The response body is a JSON object:

```
{  id: "cmolh7j230000jwrs6yh272tu",  title: "Réparation climatiseur urgent",  description: "Le climatiseur ne refroidit plus depuis hier soir.",  neededAt: "2026-12-31T12:00:00.000Z",  location: "Casablanca",  status: "OPEN",  version: 1,  createdAt: "2026-04-30T12:44:40.635Z",  updatedAt: "2026-04-30T12:44:40.635Z",  clientId: "cmolgtwg00028grsoc7gp9b5",}
```

Capture : api-2-post-create.png

5.4 PATCH /api/service-requests/[id]

200 · Mise a jour partielle

Modification partielle d'une demande ; succes.

The screenshot displays a REST client interface with the following details:

- METHOD:** PATCH
- URL:** http://localhost:3000/api/service-requests/cmon2z9h1000fusrj0nv18ak
- QUERY PARAMETERS:** (Empty)
- HEADERS:** Content-Type: application/json
- BODY:**

```
{
  "title": "SOS: projecteur installe et test son/video",
  "status": "OPEN"
}
```
- Response:** 200 OK
- Response Headers:** x-clerk-auth-reason: session-token-and-uet-missing, x-clerk-auth-status: signed-out, x-middleware-rewrite: /api/service-requests/cmon2z9h1000fusrj0nv18ak, vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch, content-type: application/json, Date: Sat, 02 May 2026 15:02:31 GMT, Connection: keep-alive, Keep-Alive: timeout=5, Transfer-Encoding: chunked
- Response Body:**

```
{
  "id": "cmon2z9h1000fusrj0nv18ak",
  "title": "SOS: projecteur installe et test son/video",
  "description": "Instabilité Wi-Fi depuis une mise à jour. Besoin diagnostic et configuration stable.",
  "neededAt": "2026-05-04T15:41:52.345Z",
  "location": "Montréal (Rosemont)",
  "status": "OPEN",
  "version": 1,
  "createdAt": "2026-05-01T15:41:52.693Z",
  "updatedAt": "2026-05-02T15:02:30.933Z",
  "clientId": "cmon2z8sv0002usrjwytidax",
}
```

Capture : api-3-patch-status.png

5.5 DELETE /api/service-requests/[id]

400 • Suppression refusee (regle metier)

Le DELETE est correctement refuse car une reservation est liee a la demande.

The screenshot shows a REST client interface with the following details:

- Request:**
 - METHOD:** DELETE
 - URL:** http://localhost:3000/api/service-requests/cmon2z98h000dusrshbl0ydgdm
 - QUERY PARAMETERS:** (empty)
 - HEADERS:** (empty)
 - Body:** XHR does not allow payloads for DELETE request.
- Response:**
 - Status:** 400 Bad Request
 - Headers:**
 - x-clerk-auth-reason: session-token-and-uaa-missing
 - x-clerk-auth-status: signed-out
 - x-middleware-rewrite: /api/service-requests/cmon2z98h000dusrshbl0ydgdm
 - vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
 - content-type: application/json
 - Date: Sat, 02 May 2026 15:13:54 GMT
 - Connection: keep-alive
 - Keep-Alive: timeout=5
 - Transfer-Encoding: chunked
 - Body:**

```
{  "error": "Suppression impossible: une reservation est liee a cette demande."}
```

Capture : api-4-delete.png

5.6 POST /api/service-requests

400 · Validation Zod echouee

Le serveur refuse une requete avec des champs invalides ; reponse avec detail des erreurs par champ.

The screenshot displays a REST client interface. The top section shows the request configuration: METHOD is POST, URL is http://localhost:3000/api/service-requests, and the body is a JSON object: {"title":"x","description":"court","neededAt":"date-invalid","clientId":"","categoryId":""}. The request is sent, and the response is shown below. The response status is 400 Bad Request. The response body is a JSON object: {"error": "Validation échouée", "details": {"title": ["Too small: expected string to have >=5 characters"], "description": ["Too small: expected string to have >=10 characters"], "neededAt": ["Invalid input: expected date, received Date"]}}. The response headers are also visible.

Request Configuration:

- METHOD: POST
- URL: http://localhost:3000/api/service-requests
- Content-Type: application/json
- Body: {"title":"x","description":"court","neededAt":"date-invalid","clientId":"","categoryId":""}

Response:

400 Bad Request

Response Headers:

- vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
- content-type: application/json
- Date: Fri, 01 May 2026 15:19:28 GMT
- Connection: keep-alive
- Keep-Alive: timeout=5
- Transfer-Encoding: chunked

Response Body:

```
{
  "error": "Validation échouée",
  "details": {
    "title": [
      "Too small: expected string to have >=5 characters"
    ],
    "description": [
      "Too small: expected string to have >=10 characters"
    ],
    "neededAt": [
      "Invalid input: expected date, received Date"
    ]
  }
}
```

Capture : api-5-validation-400.png

5.7 GET /api/service-requests/[id]/offers

200 · Offres d'une demande

Recupere toutes les offres soumises pour une demande donnee, avec le profil du prestataire.

The screenshot shows a REST client interface. The top section is for the request, with the method set to GET and the URL to `http://localhost:3000/api/service-requests/cmon2z9h1000fusrsgj0nv18ak/offers`. The bottom section shows the response, which is a 200 OK status. The response body contains two offers in a JSON array.

Request:

- METHOD: GET
- URL: `http://localhost:3000/api/service-requests/cmon2z9h1000fusrsgj0nv18ak/offers`
- HEADERS: (empty)
- BODY: (empty)

Response:

200 OK

HEADERS:

- x-clerk-auth-reason: session-token-and-ua-ut-missing
- x-clerk-auth-status: signed-out
- x-middleware-rewrite: /api/service-requests/cmon2z9h1000fusrsgj0nv18ak/offers
- vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
- content-type: application/json
- Date: Sat, 02 May 2026 15:06:23 GMT
- Connection: keep-alive
- Keep-Alive: timeout=5
- Transfer-Encoding: chunked

BODY:

```
[{"id": "cmon2z9nr000kusrsfg00zxfgo", "price": 11000, "message": "Je peux faire un diagnostic à distance (si accès routeur) + recommandations.", "status": "PENDING", "version": 1, "createdAt": "2026-05-01T15:41:52.935Z", "updatedAt": "2026-05-01T15:41:52.935Z", "requestId": "cmon2z9h1000fusrsgj0nv18ak", "providerId": "cmon2z9130006usrsrfwh01ni", "provider": {"id": "cmon2z9130006usrsrfwh01ni", "clerkId": "clerk_provider_002", "email": "pro.writer@sos-b"}}, {"id": "cmon2z9mr000jusrsfn11mha8", "price": 12000, "message": "Je peux passer pour diagnostiquer le réseau (cable, DNS, mesh) et stabiliser.", "status": "PENDING", "version": 1, "createdAt": "2026-05-01T15:41:52.899Z", "updatedAt": "2026-05-01T15:41:52.899Z", "requestId": "cmon2z9h1000fusrsgj0nv18ak", "providerId": "cmon2z9130006usrsrfwh01ni", "provider": {"id": "cmon2z9130006usrsrfwh01ni", "clerkId": "clerk_provider_002", "email": "pro.writer@sos-b"}}
```

Capture : offre d'une demande.png

5.8 POST /api/service-requests/[id]/offers

201 • Creation d'une offre

Un prestataire soumet une offre (prix + message). Statut initial PENDING, version 1.

The screenshot displays a REST client interface with the following details:

- Method:** POST
- URL:** `http://localhost:3000/api/service-requests/cmolg8ui2000e8grsafq6nybki/offers`
- Query Parameters:** None
- Headers:** `Content-Type: application/json`
- Body:**

```
{
  "price": 120,
  "message": "Je peux passer demain matin.",
  "providerId": "cmolg8ui2000e8grsafq6nybki"
}
```
- Response:** 201 Created
- Response Headers:** `vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch, content-type: application/json, Date: Thu, 30 Apr 2026 13:06:34 GMT, Connection: keep-alive, Keep-Alive: timeout=5, Transfer-Encoding: chunked`
- Response Body:**

```
{
  "id": "cmolhzogo0001jwsrcueynmc",
  "price": 120,
  "message": "Je peux passer demain matin.",
  "status": "PENDING",
  "version": 1,
  "createdAt": "2026-04-30T13:06:34.008Z",
  "updatedAt": "2026-04-30T13:06:34.008Z",
  "requestId": "cmolg8ui2000e8grsafq6nybki",
  "providerId": "cmolg8ui2000e8grsafq6nybki",
  "provider": {
    "id": "cmolg8ui2000e8grsafq6nybki",
    "name": "SuperMoms",
    "email": "supermoms@supermoms.com",
    "password": "SuperMoms123",
    "role": "client"
  }
}
```

Capture : Creation d'une offre.png

5.9 GET /api/offers/[id]

200 · Detail d'une offre

Detail complet d'une offre, incluant le prestataire et la demande associee.

The screenshot displays a REST client interface. The top section shows a GET request to the URL `http://localhost:3000/api/offers/cmon2z9lu000iusrsn1ct3unf`. The response status is **200 OK**. The response body is a JSON object containing details about an offer and its provider.

Request Headers:

- `x-clerk-auth-reason`: session-token-and-ua-ut-missing
- `x-clerk-auth-status`: signed-out
- `x-middleware-rewrite`: /api/offers/cmon2z9lu000iusrsn1ct3unf
- `vary`: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch
- `content-type`: application/json
- `Date`: Sat, 02 May 2026 15:11:46 GMT
- `Connection`: keep-alive
- `Keep-Alive`: timeout=5
- `Transfer-Encoding`: chunked

Response Body:

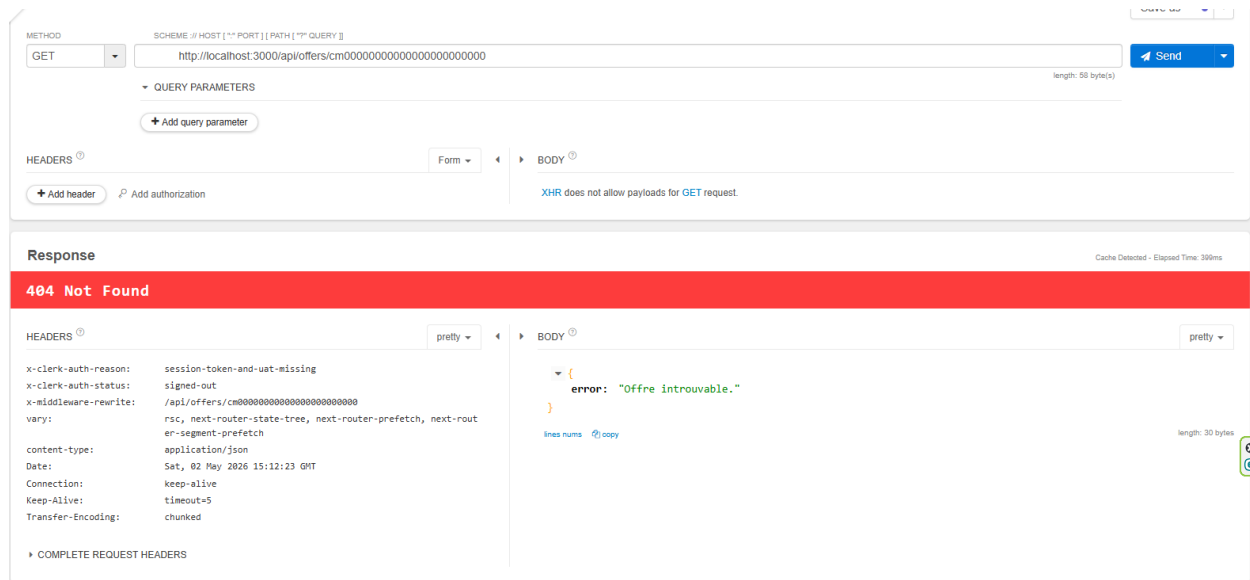
```
{
  "id": "cmon2z9lu000iusrsn1ct3unf",
  "price": 9900,
  "message": "Je peux corriger 10 pages d'ici demain 18h, avec annotations et version finale.",
  "status": "PENDING",
  "version": 1,
  "createdAt": "2026-05-01T15:41:52.866Z",
  "updatedAt": "2026-05-01T15:41:52.866Z",
  "requestId": "cmon2z9cr000eusrsbh40ht2p",
  "providerId": "cmon2z9130006usrsrfwh0lni",
  "provider": {
    "id": "cmon2z9130006usrsrfwh0lni",
    "clerkId": "clerk_provider_002",
    "email": "pro.writer@sos-besoin.test",
    "name": "Sam Correctrice",
    "role": "PROVIDER",
    "createdAt": "2026-05-01T15:41:52.119Z",
  }
}
```

Capture : detail d'une offre.png

5.10 POST /api/service-requests/[id]/offers

404 · Demande introuvable

Tentative de creer une offre sur un id inexistant ; renvoie "Demande introuvable."



Capture : Demande inexistante.png

5.11 GET /api/offers/[id]

404 · Offre introuvable

Acces a une offre inexistante ; renvoie "Offre introuvable."

The screenshot displays a REST client interface. The top section shows a POST request to the URL `http://localhost:3000/api/service-requests/cm0lg8uqr00048grs1jh7ph6v/offers`. The request body is a JSON object:

```
{  "price": 120,  "message": "Je peux passer demain matin.",  "providerId": "cm0lg8uqr00048grs1jh7ph6v"}
```

. The response section shows a **404 Not Found** status. The response headers include: `vary: rsc, next-router-state-tree, next-router-prefetch, next-router-segment-prefetch`, `content-type: application/json`, `Date: Thu, 30 Apr 2026 12:51:22 GMT`, `Connection: keep-alive`, `Keep-Alive: timeout=5`, and `Transfer-Encoding: chunked`. The response body is a JSON object:

```
{  "error": "Demande introuvable."}
```

. The interface also shows a 'Send' button for the request and a 'Cache Detected - Elapsed Time: 50ms' message.

Capture : offre inexistante.png

6. Fonctionnement du panier

L'Epic C du projet est un **panier client persistant** qui permet a un client connecte de regrouper plusieurs offres acceptees avant de proceder au paiement. La persistance est cote serveur via deux nouveaux modeles Prisma (*Cart* et *CartItem*), ce qui garantit que le contenu du panier reste disponible meme apres deconnexion ou changement d'appareil.

6.1 Flux pas a pas

1. Ajout au panier — Depuis la page de detail d'une demande, un bouton *Ajouter au panier* sur chaque offre PENDING declenche la Server Action `addToCart(offerId)`. Le serveur cree (ou recupere) le *Cart* de l'utilisateur, valide qu'il n'existe pas deja un *CartItem* pour cette offre, puis insere une nouvelle ligne. Un badge avec un compteur s'affiche dans le header.

2. Consultation du panier — La page `/cart` appelle `getCart()` qui charge le panier avec ses items et les offres / demandes associees. Chaque ligne affiche le titre de la demande, le prestataire, le prix unitaire et un bouton *Supprimer*.

3. Suppression d'un item — `removeFromCart(itemId)` verifie que l'item appartient bien au panier de l'utilisateur connecte avant suppression (controle d'autorisation au niveau Server Action).

4. Calcul du total — Le sous-total est la somme des `price * quantity` de chaque *CartItem*. Une **commission plateforme de 10%** est ajoutee. Le total final = sous-total + frais.

5. Confirmation et paiement — Le bouton *Confirmer et proceder au paiement* appelle `confirmCart()` qui transforme chaque *CartItem* en *Booking* dans une transaction Prisma (`prisma.$transaction`) : creation des bookings, mise a jour des offres `ACCEPTED/REJECTED`, mise a jour de la demande en `FILLED`, et vidage du panier. Tout reussit ou tout echoue (atomicite ACID).

6. Vider le panier — `clearCart()` supprime tous les *CartItem* de l'utilisateur en une seule requete (`deleteMany`).

6.2 Modeles Prisma ajoutes

```
model Cart {
  id String @id @default(cuid())
  userId String @unique
  user User @relation(...)
  items CartItem[]
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model CartItem {
  id String @id @default(cuid())
  cartId String
  offerId String
  quantity Int @default(1)
  cart Cart @relation(...)
  offer Offer @relation(...)
  @@unique([cartId, offerId])
}
```

```
}
```

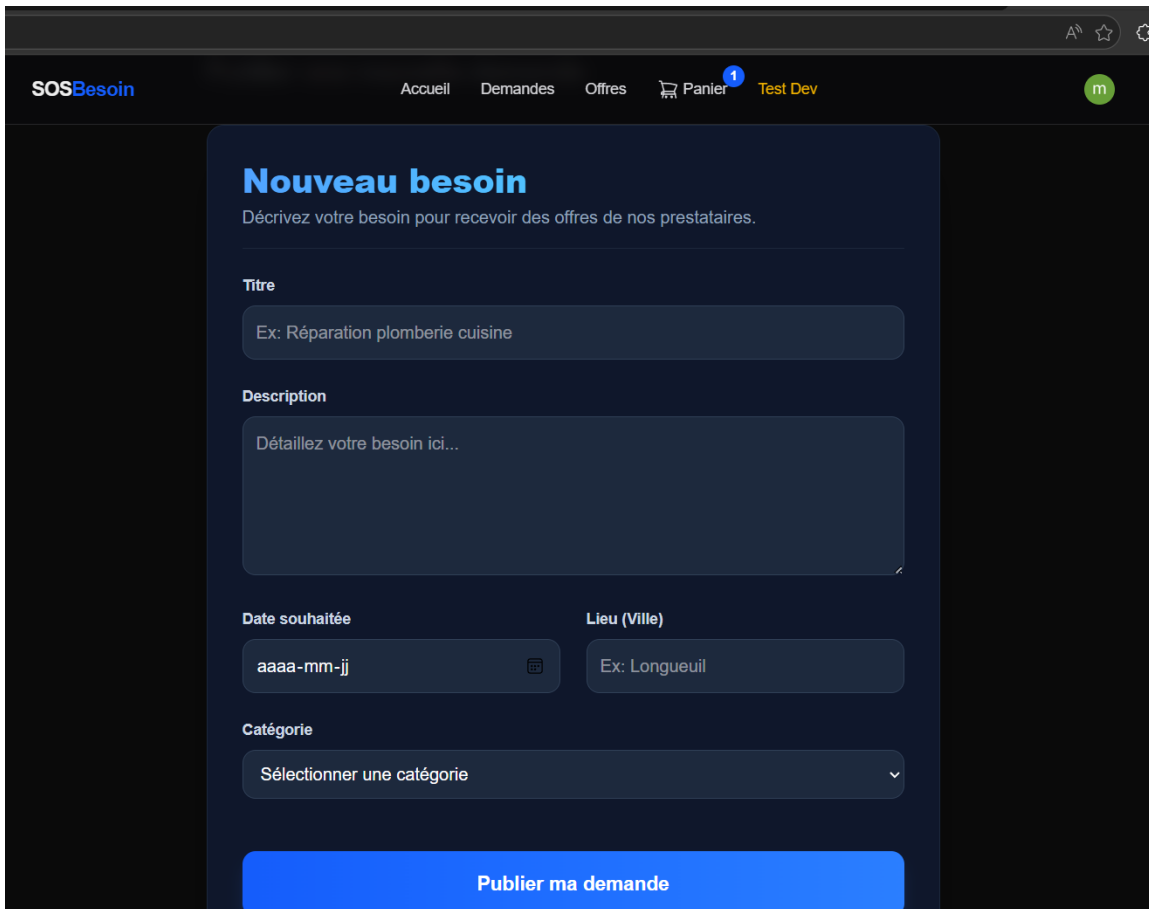
Migration : prisma/migrations/20260430052150_add_cart_cartitem/.

7. Captures d'ecran - Interface utilisateur

Les captures suivantes illustrent les écrans principaux de l'application. L'interface s'adapte automatiquement au mode clair / sombre du système de l'utilisateur grâce aux classes Tailwind dark:. La pagination est implementee dans `src/components/Pagination.tsx` et la barre de recherche (`SearchBar.tsx`) est synchronisee avec les `searchParams` de l'URL pour permettre des liens partageables.

7.1 Formulaire CRUD - Creation d'une demande

Page "Nouveau besoin" avec les champs valides par Zod : titre, description, date souhaitee, lieu et categorie. Le bouton de soumission affiche un spinner pendant la creation.



Capture : image (48).png

7.2 Formulaire CRUD - Edition (avec champs caches id + version)

Page "Modifier la demande" avec le formulaire pre-rempli a partir des donnees existantes. Deux champs `<input type="hidden">` sont presents dans le DOM : id et version (verrou optimiste). Si la demande a ete modifiee entre temps par un autre utilisateur, l'updateMany echoue et un message de conflit est retourne.

[← Retour au detail](#)

Modifier la demande

Mettez à jour les informations de votre demande de service.

Titre
reparation arbalette

Description
besoin d'un spécialiste pour reparer mon arbalette "solution 2000"

Date souhaitée
05/08/2026

Lieu (Ville)
St-Lazare

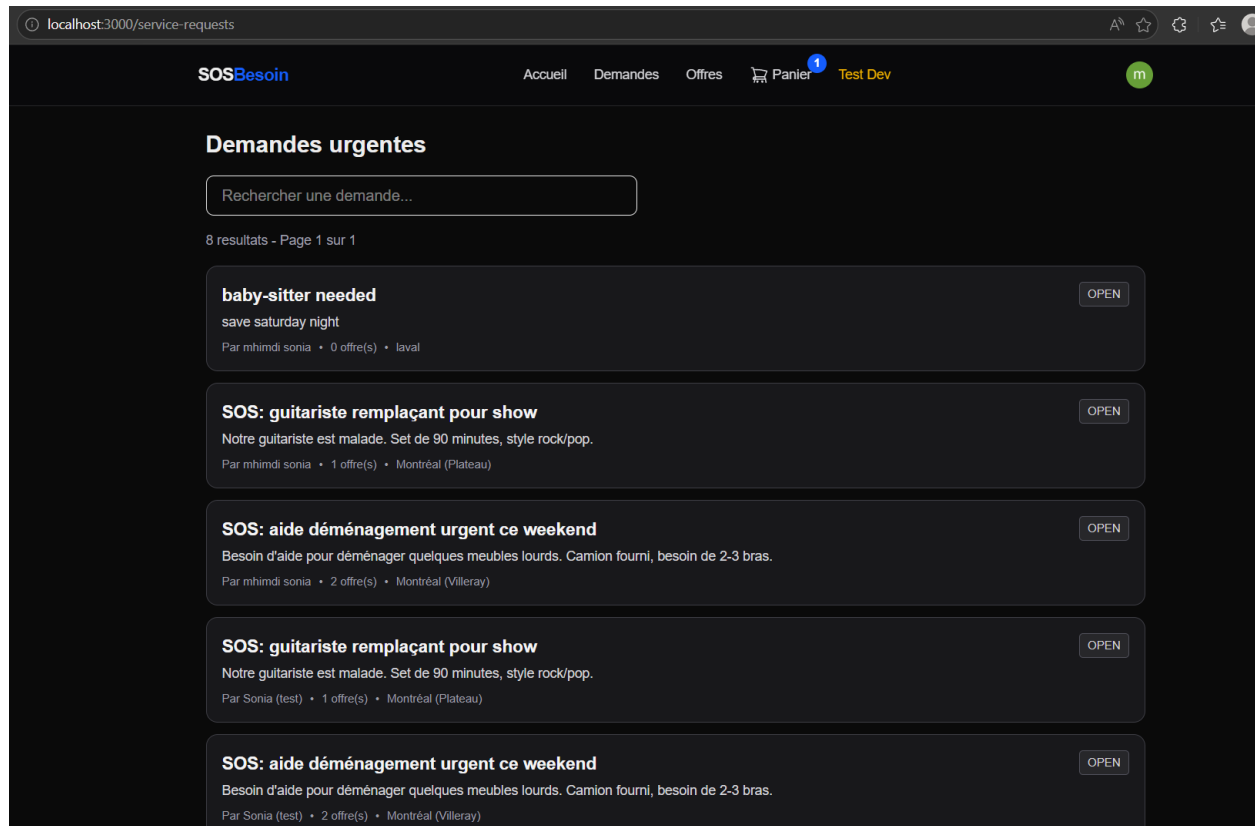
Catégorie
Maison

Mettre à jour la demande

Capture : Capture d'écran 2026-05-03 120707.png

7.3 Liste paginee des demandes (avec SearchBar)

Affichage des demandes urgentes, avec barre de recherche debounee (300 ms) synchronisee avec les searchParams de l'URL. Le compteur de resultats et la pagination s'adaptent automatiquement aux filtres.



Capture : image (49).png

7.4 Etat vide - Aucune demande trouvee

Recherche avec un terme inexistant (q=zzzzz) : la liste affiche le message "Aucune demande trouvee." et le compteur indique 0 resultat.

Demandes urgentes

 × ...

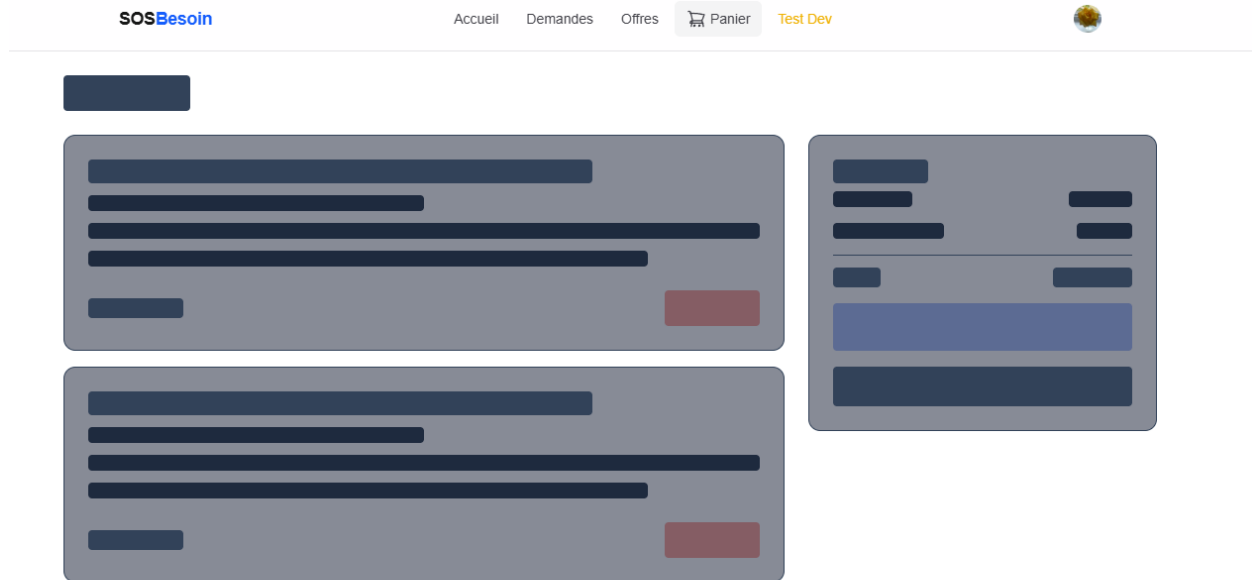
0 resultat - Page 1 sur 1

Aucune demande trouvee.

Capture : Capture d'écran 2026-05-03 101650.png

7.5 Etat de chargement - Squelettes (loading.tsx)

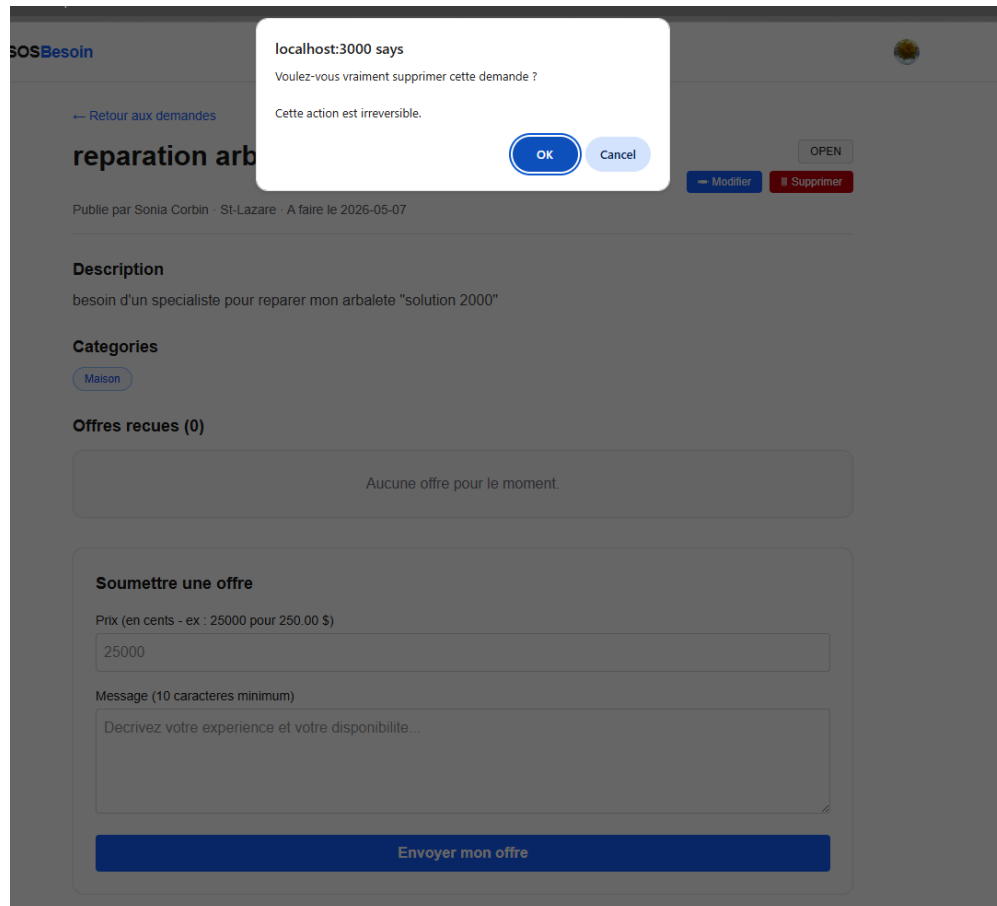
Squelette de la page panier (rectangles gris pulsants via `animate-pulse`) qui s'affiche pendant que `getCart()` charge les données. Implemente dans `src/app/cart/loading.tsx`. Un squelette equivalent existe pour la liste des demandes (`src/app/service-requests/loading.tsx`).



Capture : Capture d'écran 2026-05-03 105550.png

7.6 Confirmation de suppression

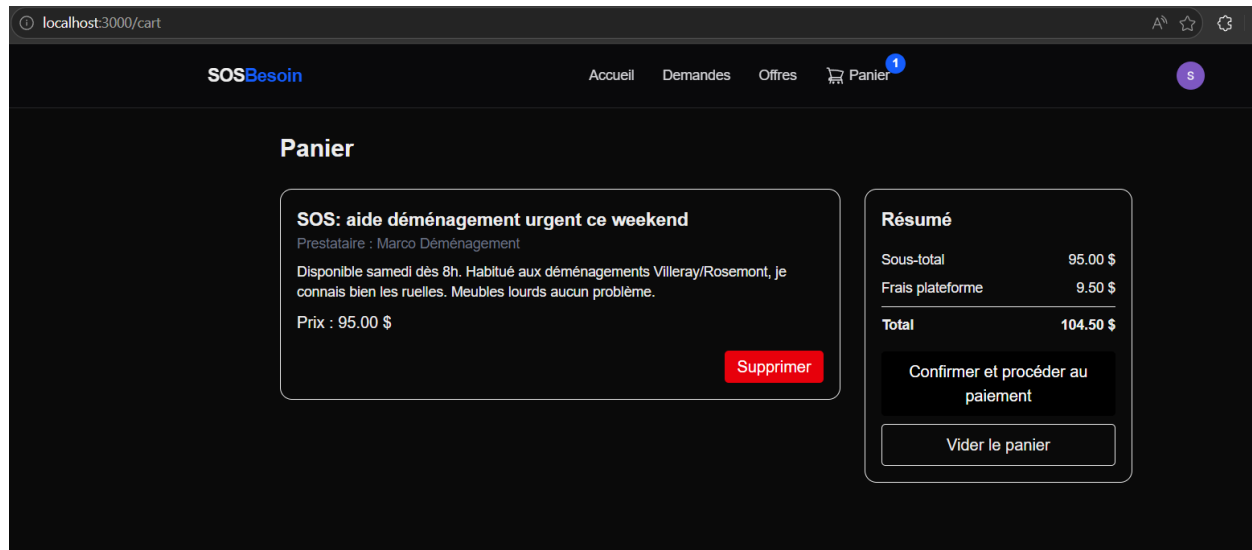
Popup native du navigateur "Voulez-vous vraiment supprimer cette demande ? Cette action est irreversible." avec boutons OK / Cancel. Implementee via `window.confirm()` dans le composant `DeleteRequestButton.tsx`. Si l'utilisateur confirme, l'action serveur `deleteRequestAction(id)` est appelee et l'utilisateur est redirige vers la liste.



Capture : Capture d'écran 2026-05-03 120722.png

7.7 Panier - Vue principale (Epic C)

Page /cart avec : badge "1" sur l'icone panier dans le header, carte decrivant l'offre ajoutée (titre de la demande, prestataire, description, prix), bouton Supprimer rouge, et resume des couts a droite (sous-total 95.00 \$, frais plateforme 10% = 9.50 \$, total 104.50 \$). Boutons "Confirmer et proceder au paiement" et "Vider le panier".



Capture : image (47).png

8. Bonus - Pagination par curseur

Le bonus retenu est l'implementation d'une **pagination par curseur** (cursor-based pagination) en plus de la pagination par offset deja en place. Les deux approches coexistent dans le projet et utilisent les memes filtres.

8.1 Implementation

- Fonction `getCursorPaginatedServiceRequests()` ajoutee dans `src/lib/requetes/serviceRequests.ts`.
- Page de demo dediee : `src/app/service-requests/cursor/page.tsx`.
- URL : `/service-requests/cursor?cursor=lastId&q=...&status=OPEN`
- Page existante `/service-requests` conservee en mode **offset**.

Extrait du code Prisma (curseur)

```
const items = await prisma.serviceRequest.findMany({
  where,
  take: take + 1, // +1 pour detecter hasMore
  ...(params.cursor ? {
    cursor: { id: params.cursor },
    skip: 1, // ne PAS reinclure l'item du curseur
  } : {}),
  orderBy: [{ createdAt: "desc" }, { id: "desc" }],
  include: { client: true, categories: ..., _count: ... },
});
```

Astuce. On demande `take + 1` elements : si on en recoit `take + 1`, on sait qu'il existe une page suivante (`hasMore = true`) et le `nextCursor` est l'id du dernier element retourne. Le `orderBy` inclut un tie-breaker stable sur `id` pour eviter les sauts d'elements lors d'egalites sur `createdAt`.

8.2 Comparaison Offset vs Curseur

Critere	Offset (skip / take)	Curseur (cursor / take)
Requete SQL generee	OFFSET N LIMIT M	WHERE id apres cursor + LIMIT M
Performance gros volumes	Degradé avec N (scan + rejet des N premieres lignes)	Stable : utilise un index unique sur id.
Acces a une page N	Direct (page=42).	Impossible : navigation sequentielle uniquement.
Retour en arriere	Trivial (page--).	Possible mais demande une pile de curseurs cote client.
Insertion concurrente	Risque de duplication ou saut d'elements.	Robuste : un nouvel element n'affecte pas la suite.
Affichage du total	Naturel (count() en parallele).	Plus couteux : count() reste necessaire.
Implementation	Simple : skip / take.	Subtile : tie-breaker + take+1 + skip:1.
Cas d'usage	Tableaux d'admin, navigation arbitraire.	Feeds, scroll infini, gros volumes.

8.3 Conclusion

Pour SOS-BESOIN, la pagination par **offset** reste pertinente sur /service-requests car elle permet d'afficher un total et de sauter directement a une page donnee. La pagination par **curseur**, exposee sur /service-requests/cursor, devient le meilleur choix des que la liste depasse plusieurs milliers d'enregistrements ou pour un futur scroll infini.